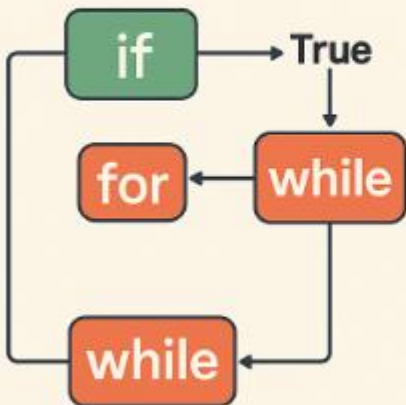


PART 2 — Control Flow

The easiest way to understand Control Flow is:

Control flow = deciding WHERE the program goes and HOW MANY TIMES it executes something.

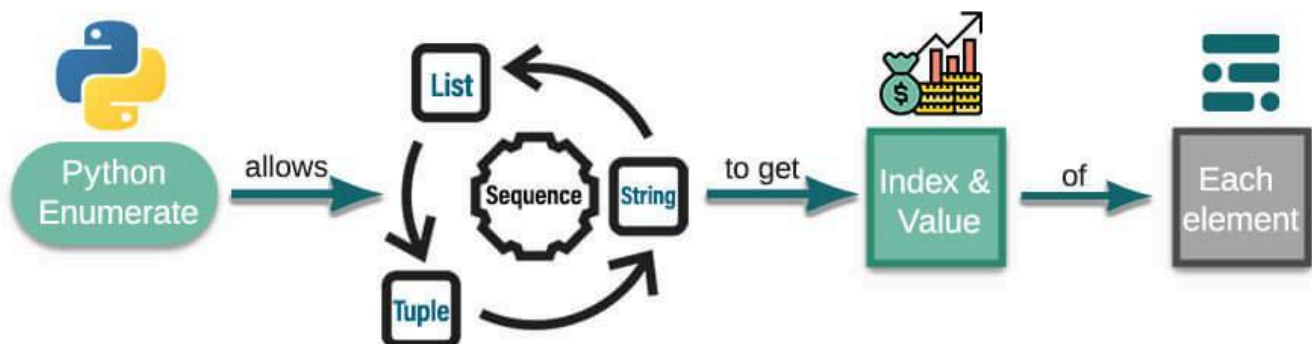
Learn Control Flow in Python: If-Else, While and For Loops Made Easy

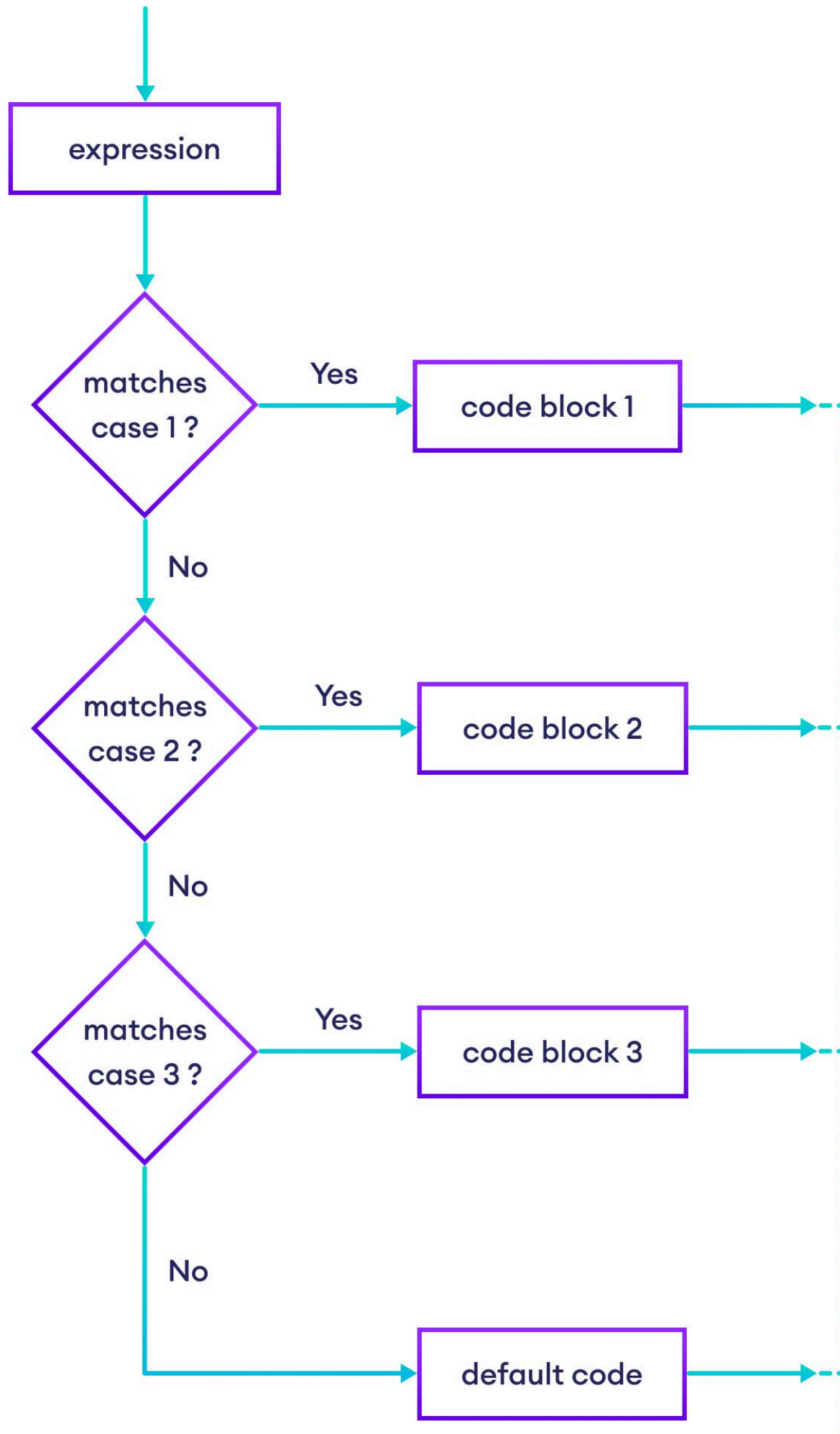


```
for i in range(5):  
    if i % 2 == 0:  
        continue  
    print(i)
```

for i in range(5):	1
if i % 2 == 0:	3
continue	4
print(i)	

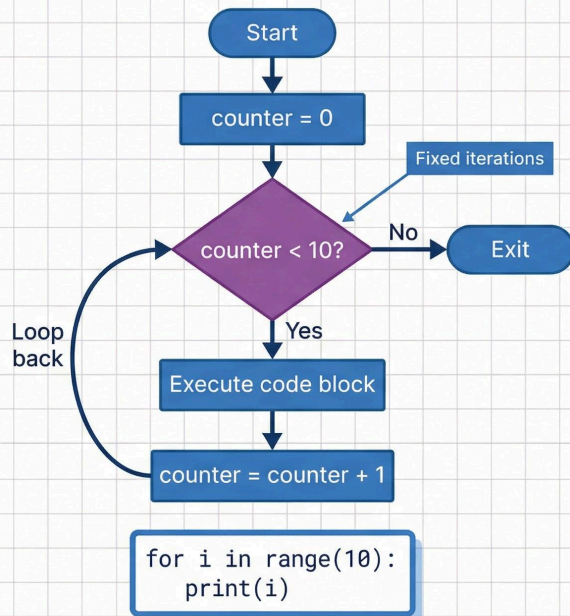
Python Enumerate



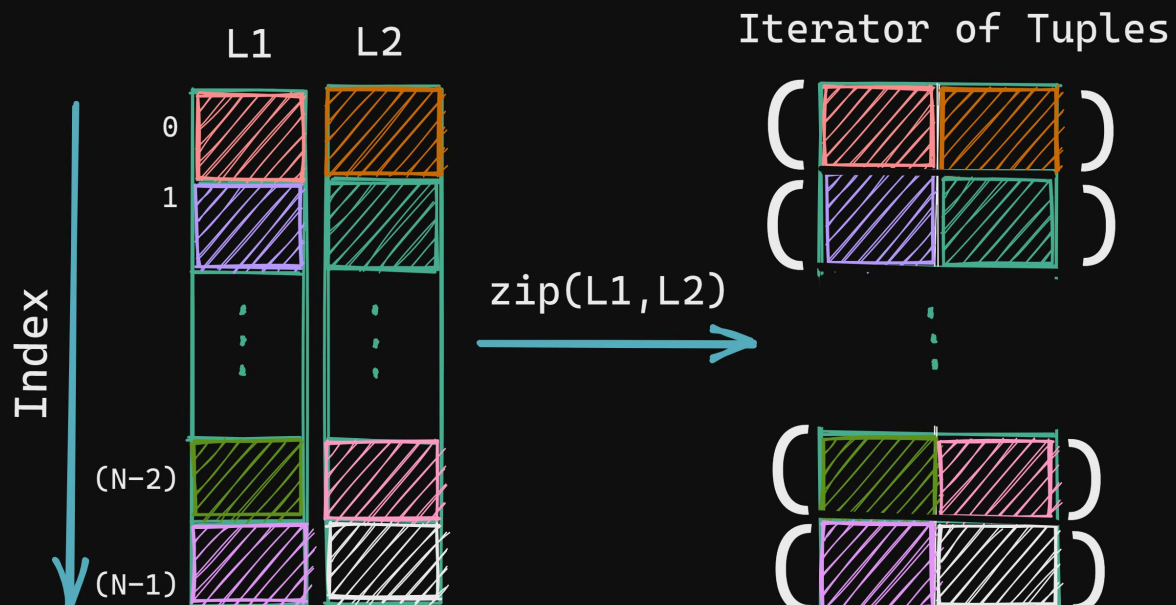
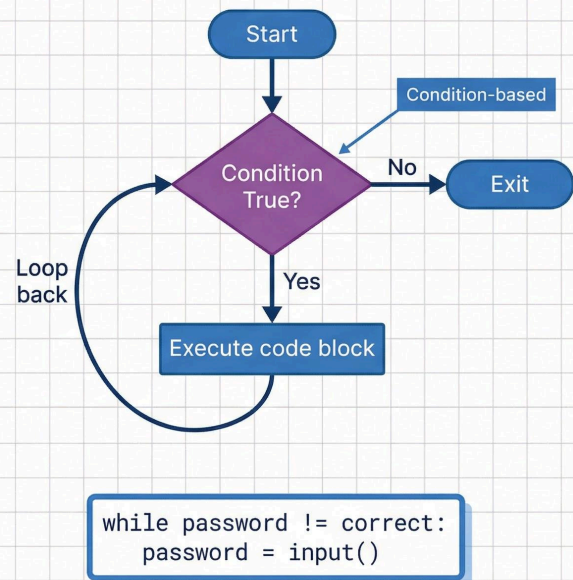


match
terminates

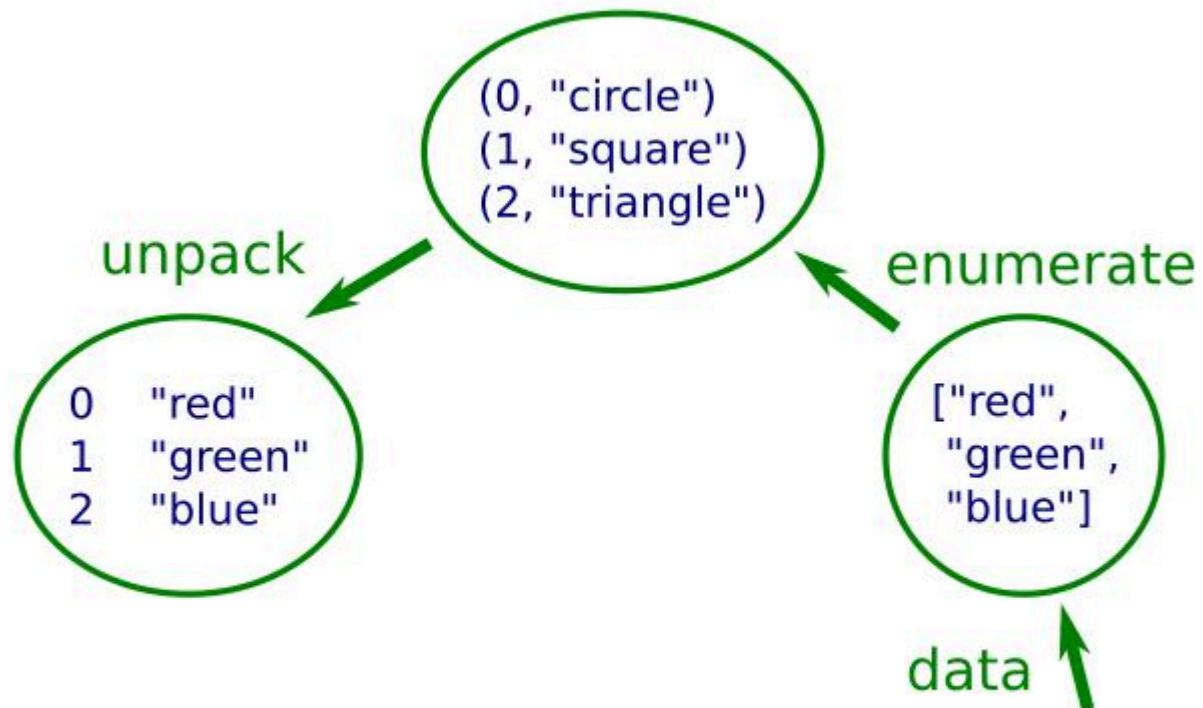
FOR LOOP



WHILE LOOP

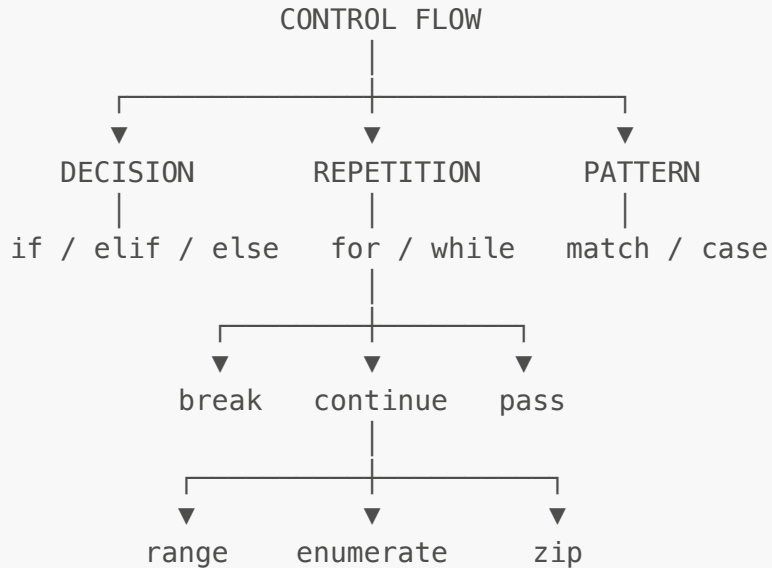


By Bala Priya



for i, c in enumerate(colors)

🧠 THE MASTER MAP



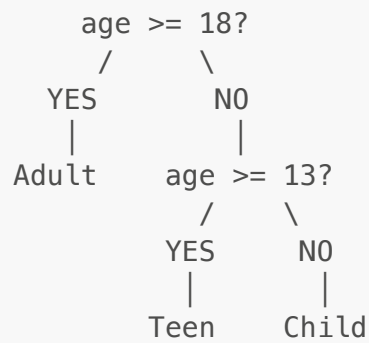
1. if / elif / else

Used when your program needs to **make a decision**.

```
age = 25
```

```
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teenager")
else:
    print("Child")
```

Think:



Important

Python checks conditions **from top to bottom**.

```
if condition1:
    ...
elif condition2:
    ...
elif condition3:
    ...
else:
    ...
```

Once one condition is **True**, the remaining branches are skipped.

🔥 Mental model

if	→ first possibility
elif	→ another possibility
else	→ everything remaining

2. for loops

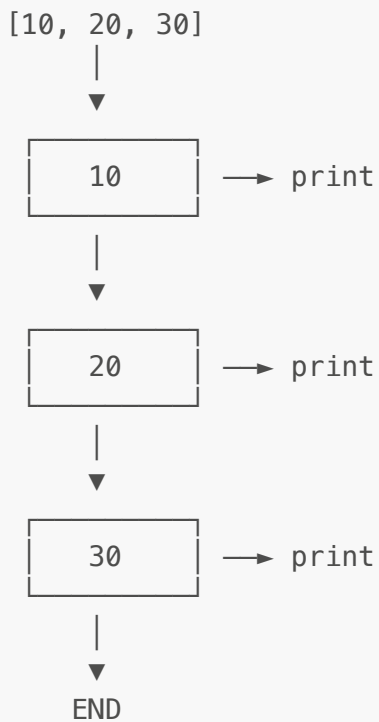
A **for** loop means:

Take items one by one from an iterable.

```
numbers = [10, 20, 30]

for number in numbers:
    print(number)
```

Visual:



A **for** loop works with **iterables**:

```
list
tuple
string
set
dict
range
generator
file
...
```

Example:

```
for char in "Python":
    print(char)
```

3. while loops

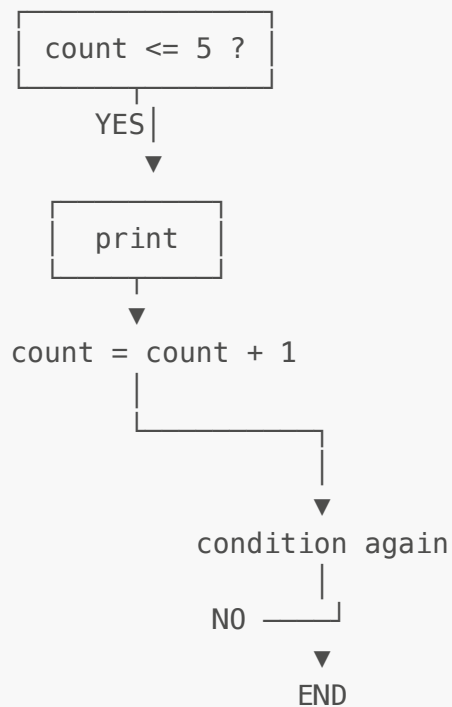
A **while** loop means:

Keep executing while the condition is True.

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Visual:



⚠ Infinite loop

```
count = 1

while count <= 5:
    print(count)
```

Problem:

```
count never changes
    ↓
condition stays True
    ↓
FOREVER ∞
```

Mental model

```
for → iterate over something
while → repeat while condition is true
```

4. break / continue / pass

These are **flow-control tools**.

break

Immediately leave the loop.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```

Visual:

```
0 → 1 → 2 → 3 → 4 → 5
                      |
                      break
                      |
                      ▼
                      END
```

continue

Skip this iteration and move to the next one.

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output:

```
0  
1  
3  
4
```

Visual:

```
0 → print  
1 → print  
2 → SKIP → next iteration  
3 → print  
4 → print
```

pass

Do nothing.

```
for i in range(5):  
    if i == 2:  
        pass  
    print(i)
```

`pass` does **not** skip the iteration.

It literally means:

```
"There's nothing to do here."
```

Useful when writing incomplete code:

```
def future_function():  
    pass
```

★ Remember

```
break    → EXIT  
continue → SKIP  
pass     → DO NOTHING
```

5. range()

`range()` generates a sequence of numbers.

One argument

```
range(5)
```

means:

```
0 1 2 3 4
```

Not `0 → 5`.

Two arguments

```
range(2, 6)
```

means:

```
2 3 4 5
```

Three arguments

```
range(1, 10, 2)
```

means:

```
1 3 5 7 9
```

Visual:

```
range(start, stop, step)
  |      |      |
  |      |      +-- jump
  |      +-----+-- excluded
  +-----+-----+-- included
```

Reverse

```
range(5, 0, -1)
```

```
5 → 4 → 3 → 2 → 1
```

🔥 Important

In Python 3:

```
range(1000000000)
```

doesn't create a billion integers as a list in memory.

It represents the sequence efficiently.

6.enumerate()

Suppose:

```
names = ["Alice", "Bob", "Charlie"]
```

You want both:

index + value

Without `enumerate()`:

```
for i in range(len(names)):
    print(i, names[i])
```

With `enumerate()`:

```
for i, name in enumerate(names):
    print(i, name)
```

Visual:

names



0	Alice
1	Bob
2	Charlie

Output:

```
0 Alice
1 Bob
2 Charlie
```

You can choose the starting index:

```
for i, name in enumerate(names, start=1):
    print(i, name)
```

Output:

```
1 Alice
2 Bob
3 Charlie
```

Mental model

```
enumerate()
  ↓
(index, value)
```

7. `zip()`

`zip()` lets you iterate through multiple iterables **in parallel**.

```
names = ["Alice", "Bob", "Charlie"]
scores = [90, 85, 88]

for name, score in zip(names, scores):
    print(name, score)
```

Visual:

names		scores
Alice	→	90
Bob	→	85
Charlie	→	88

Output:

```
Alice 90
Bob 85
Charlie 88
```

⚠ Important

`zip()` stops at the **shortest iterable**.

```
names = ["Alice", "Bob", "Charlie"]
scores = [90, 85]
```

Then:

```
list(zip(names, scores))
```

gives:

```
[
    ("Alice", 90),
    ("Bob", 85)
]
```

Charlie is not included.

Mental model

```
zip(A, B)

A1 — B1
A2 — B2
A3 — B3
```

8. match / case

`match/case` is Python's pattern-matching syntax.

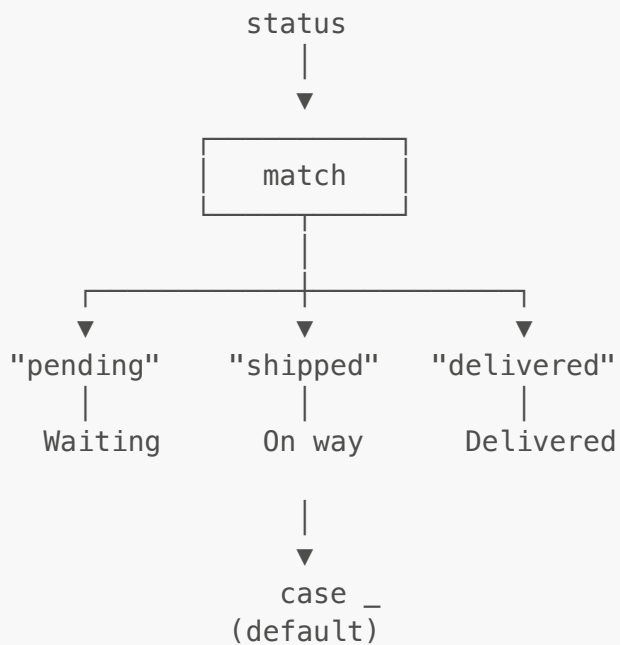
It is useful when you have multiple possible patterns/values.

```
status = "shipped"

match status:
    case "pending":
        print("Waiting")
    case "shipped":
        print("On the way")
    case "delivered":
        print("Delivered")
```

```
case _:
    print("Unknown")
```

Visual:



The `_` acts as a catch-all pattern.

if/elif vs match/case

```

if / elif / else
  ↓
general conditions

match / case
  ↓
patterns / structured matching
```

For simple value comparisons, both can sometimes work.



The most important comparison

for vs while

```

FOR
├── "Give me each item"
```

```
WHILE
├
└ "Keep going while this is true"
```

Example:

```
for x in [1, 2, 3]:
    print(x)
```

You already have something to iterate over.

Whereas:

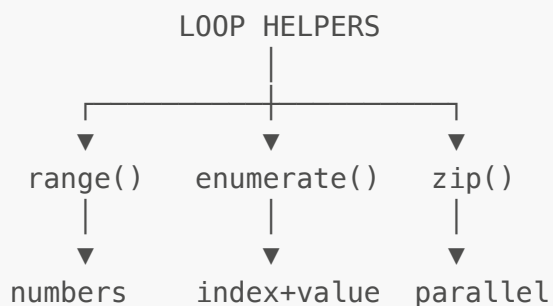
```
x = 1

while x <= 3:
    print(x)
    x += 1
```

You control the condition.

range, enumerate, zip

These three are worth learning together:



Example

```
names = ["A", "B", "C"]
scores = [90, 80, 70]

for i, (name, score) in enumerate(zip(names, scores), start=1):
    print(i, name, score)
```

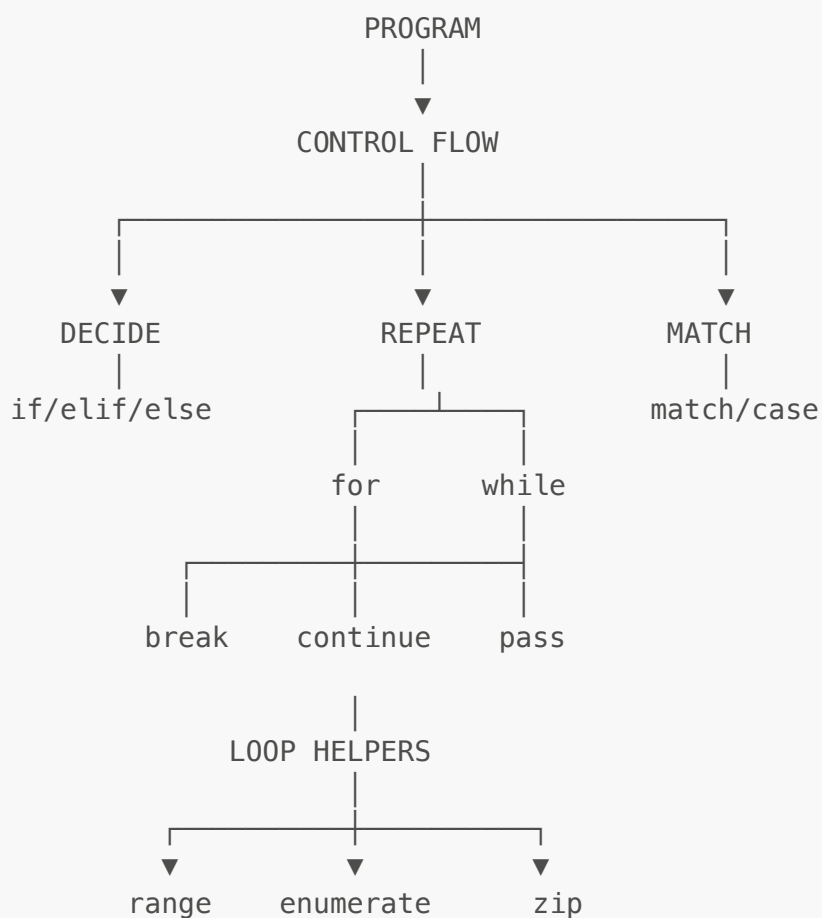

Output:

```
1 A 90
2 B 80
3 C 70
```

This single example combines:

```
enumerate
+
zip
+
for
```

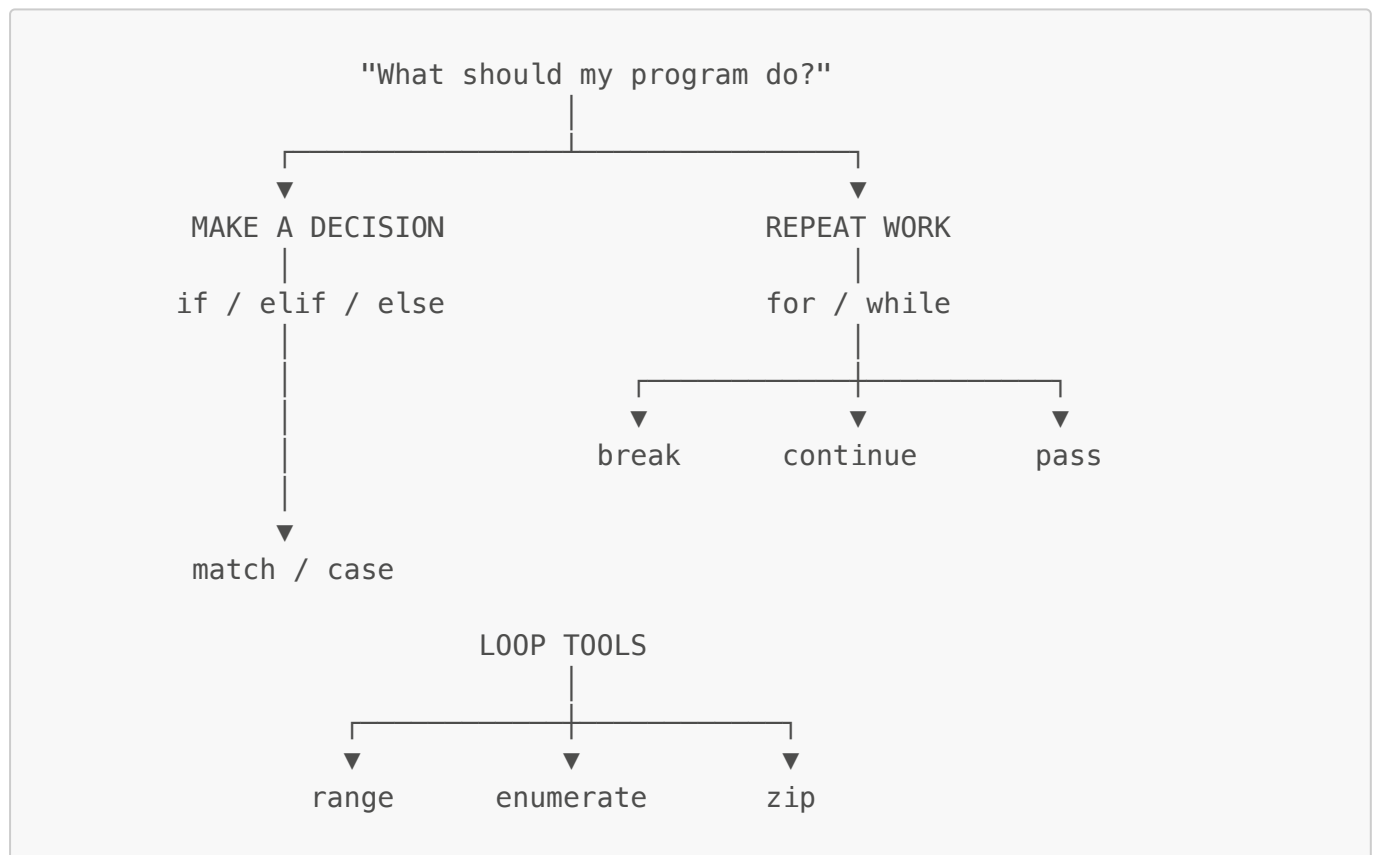
🔥 MASTER CONTROL-FLOW MAP



★ Memorize this cheat sheet

Concept	Mental model
<code>if</code>	Make a decision
<code>elif</code>	Try another condition
<code>else</code>	Everything remaining
<code>for</code>	Iterate over items
<code>while</code>	Repeat while True
<code>break</code>	Exit loop
<code>continue</code>	Skip iteration
<code>pass</code>	Do nothing
<code>range()</code>	Generate number sequence
<code>enumerate()</code>	Index + value
<code>zip()</code>	Multiple iterables together
<code>match/case</code>	Pattern matching

🎯 One final mental picture



The key idea: Part 1 taught you **what data is**. Part 2 teaches you **what your program does with that data**.

Next, the natural progression is **PART 3 — Functions**: function definition, parameters, arguments, return, scope, `*args`, `**kwargs`, lambda, recursion, first-class functions, and then the decorators/closures from

your earlier list.